Railway level crossing in mCRL2

Diogo and Rodrigo Gomes de Araújo
up202510267, up202210267

November 2026

Introduction

This report was written for the first assignment of the *Systems Verification* course at *Faculdade de Ciências da Universidade do Porto* (FCUP). It presents a model of a railway level crossing, describing the interaction between a controller, a gate, and one or more trains that wish to cross. We then formalise a set of properties that verify its correctness and check them in mCRL2.

Task 1

In this task we modelled a single train passing through the crossing.

Components

The model contains the following three processes.

$$\begin{aligned} \text{Train} &= \text{approach} \cdot \text{grant} \cdot \text{enter} \cdot \text{exit} \cdot \text{leave} \cdot \text{Train} \\ \text{Gate} &= \text{lower} \cdot \text{down} \cdot \text{raise} \cdot \text{up} \cdot \text{Gate} \\ \text{Controller} &= \text{approach} \cdot \text{lower} \cdot \text{down} \cdot \text{grant} \cdot \text{leave} \cdot \text{raise} \cdot \text{up} \cdot \text{Controller} \end{aligned}$$

Each one consists of a loop over a finite set of actions. The `Controller` enforces the ordering between the `Gate` and the `Train`'s actions. It allows the train to enter (`grant`) after the gate is `down` and raises the gate after the train `leaves`.

Composition

Every interaction is modelled as a pair of complementary actions (e.g. `approach_train` and `approach_controller`) that `comm` merges into one synchronised action. `allow` then blocks the complementary actions from occurring independently.

```mcrl2
hide({ lower, grant, leave, raise },
allow({ approach, lower, down, grant, leave, raise, up, enter, exit },
comm({ approach_controller | approach_train -> approach, ... },
      TRAIN || GATE || CONTROLLER )));
```

For inspection, we split the actions into messages and physical events. Messages are internal handshakes between the components and cannot be observed from the outside, so we hide them:

- **`lower`, `raise`:** the commands to the gate;
- **`grant`:** the permission to cross;
- **`leave`:** the train's notification that it left.

Physical events, on the other hand, are what someone standing near the crossing would observe, so they remain visible:

- **`approach`:** the train arriving;
- **`enter`, `exit`:** being on the crossing;
- **`down`, `up`:** the gate reaching its position.

Inspection

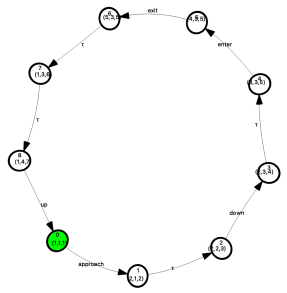


Figure 1: Full LTS

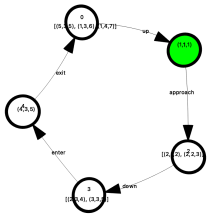


Figure 2: Reduced LTS

The full LTS (Figure 1) is a single cycle of 9 states, four of whose transitions are τ (the hidden messages). None of these τ-steps removes a choice, so branching bisimulation collapses them, leaving a cycle of 5 states made only of the physical events (Figure 2). This is the externally observable behaviour of the train passing through the crossing.

Properties

We check the composed model before minimisation. In the formulas, $-$ denotes any action and $-a$ any action other than a.

Property	Formula	Result
Safety	$[-^*;\mathrm{enter};(-\mathrm{exit})^*;\mathrm{up}]\ \mathrm{false} \wedge [(-\mathrm{down})^*;\mathrm{enter}]\ \mathrm{false} \wedge$ $[-^*;\mathrm{up};(-\mathrm{down})^*;\mathrm{enter}]\ \mathrm{false}$	true
Reachability	$[-^*;\mathrm{exit}]\langle -^*;\mathrm{up}\rangle\ \mathrm{true}$	true
Liveness	$[-^*;\mathrm{approach}]\langle -^*;\mathrm{exit}\rangle\ \mathrm{true}$	true
Deadlock freedom	$[-^*]\langle -\rangle\ \mathrm{true}$	true

The model satisfies all four properties.

- For *safety*, the train is never on the crossing while the gate is open: once a train has entered (`enter`), the gate cannot be raised (`up`) until the train exits (`exit`). Furthermore, a train can only enter if the gate has been lowered (`down`) since it was last raised, ensuring that the gate remains closed whenever the train is on the crossing.
- For *reachability*, after every `exit`, the gate can eventually reach `up`. Both events remain visible after hiding, so the formula checks the observable return to the open state.
- For *liveness*, whenever a train approaches (`approach`), it can eventually exit (`exit`), ensuring that it is not permanently prevented from completing its passage.
- Finally, *deadlock freedom* requires an outgoing transition from every reachable state, so the system cannot get stuck.

Broken Variant

In `1/variants/grant_before_down.mcrl2`, the controller issues the grant immediately after ordering the gate to lower, without waiting for the `down` acknowledgement:

```
1  1      approach_controller                                    mcrl2
2    -    . lower_controller . down_controller
```

```
3    -    . grant_controller
   2 +    . lower_controller . grant_controller
   3 +    . down_controller
```

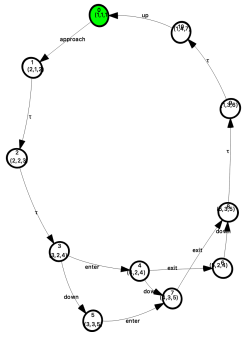


Figure 3: Broken LTS

The first-cycle conjunct of the safety property, $[(-\text{down})^*; \text{enter}]$ false, requires a `down` before the first `enter`. It fails in Figure 3 with the following counterexample:

approach . lower . grant . enter

Since `lower` and `grant` are hidden, the observable trace is just `approach, enter`: the train enters with no `down` before it, so the formula is violated. The gate process is still waiting to fire `down_gate`, because after `lower` the train and gate run independently and the train wins the race.

The other three properties remain true: the train can still exit, the gate can still return to `up`, and the system does not deadlock. This separates safe entry from the ability to keep making progress.

Task 2

In this task the model allows multiple trains to pass through the crossing one at a time.

Model with data

$\text{Train}(t)$ and Gate are the same as in Task 1, with the train's actions carrying its identity. The controller keeps a queue Q of the trains that approached and did not leave yet, whether the crossing c is `busy` or `free`, and the gate state g:

$$\begin{aligned}\text{Controller}(Q, c, g) = \sum_{t \in \text{ Id}} (t \notin Q) \to \text{approach}(t) \,.\, \text{Controller}(Q \triangleleft t, c, g) \\ +(Q \neq [\,] \wedge g = \text{closed} \wedge c = \text{free}) \to \text{grant}(\text{head}(Q)) \,.\, \text{Controller}(Q, \text{busy}, g) \\ +(Q \neq [\,] \wedge c = \text{busy}) \to \text{leave}(\text{head}(Q)) \,.\, \text{Controller}(\text{tail}(Q), \text{free}, g) \\ +(g = \text{open} \wedge Q \neq [\,]) \to \text{lower} \,.\, \text{Controller}(Q, c, \text{closing}) \\ +(g = \text{closing}) \to \text{down} \,.\, \text{Controller}(Q, c, \text{closed}) \\ +(g = \text{closed} \wedge Q = [\,] \wedge c = \text{free}) \to \text{raise} \,.\, \text{Controller}(Q, c, \text{opening}) \\ +(g = \text{opening}) \to \text{up} \,.\, \text{Controller}(Q, c, \text{open})\end{aligned}$$

where $Q \triangleleft t$ appends t to the end of Q. We chose a queue over a set so that only the first train in line can be granted. No train is overtaken, which matters for liveness.

Invariant. Q always holds exactly the trains that approached but did not leave yet: `approach(t)` is the only step that adds a train and `leave` the only one that removes it. c is `busy` exactly while one train is between `grant` and `leave`. Since `grant` needs $c =$ free, only one train crosses at a time, and since `raise` needs Q empty and $c =$ free, the gate is never raised while a train is near.

Properties

The checked model has three trains, with $\mathrm{Id} = \{t_1, t_2, t_3\}$. Formulas containing t are universally quantified over $t \in \mathrm{Id}$; formulas containing both t and t' are checked for every distinct pair. All properties are checked on the composed model before minimisation.

Property	Formula	Result
1. Safety	$[-^*; \mathrm{enter}(t); (-\mathrm{leave}(t))^*; \mathrm{up}]\ \mathrm{false}$	true
2. Mutual exclusion	$[-^*; \mathrm{enter}(t); (-\mathrm{exit}(t))^*; \mathrm{enter}(t')]\ \mathrm{false}$	true
3. No premature raise	$[-^*; \mathrm{approach}(t); (-\mathrm{leave}(t))^*; \mathrm{raise}]\ \mathrm{false}$	true
4. Deadlock freedom	$[-^*]\langle - \rangle\ \mathrm{true}$	true
5a. Liveness (reach)	$[-^*; \mathrm{approach}(t); (-\mathrm{enter}(t))^*]\langle -^*; \mathrm{enter}(t)\rangle\ \mathrm{true}$	true
5b. Liveness (order)	$[-^*; \mathrm{approach}(t); (-\mathrm{enter}(t))^*; \mathrm{enter}(t'); (-\mathrm{enter}(t))^*; \mathrm{enter}(t')]\ \mathrm{false}$	true
6. Road availability	$[-^*]\langle -^*; \mathrm{up}\rangle\ \mathrm{true}$	true
7. Infinite opening	$[-^*]\mu X \cdot (\langle - \rangle\ \mathrm{true} \wedge [-\mathrm{up}]X)$	false
8. Non-vacuity	$[-^*]\langle -^*; \mathrm{enter}(t)\rangle\ \mathrm{true} \wedge [-^*]\langle -^*; \mathrm{raise}\rangle\ \mathrm{true} \wedge [-^*]\langle - \rangle\ \mathrm{true}$	true

Safety (1–3)

The gate cannot reach `up` between a train's `enter` and `leave` (1), and no second train can enter before the first exits (2). The controller also cannot issue `raise` after a train approaches and before it leaves (3).

Entry with the gate fully down follows from the controller's `grant` guard and the train's action order: `grant` requires the state `closed`, and `enter` follows `grant`. If a train approaches while the gate is already opening, the gate may still finish with `up`. The controller then lowers it again and waits for `down` before granting access.

Liveness (4–5)

Every reachable state has an outgoing transition (4). While a train waits, its entry remains reachable (5a), and no other train can enter twice before it (5b). The latter condition excludes indefinite overtaking by repeatedly serving another train.

The FIFO queue gives a stronger structural argument: at most $N - 1$ trains are ahead of a waiting train, and later arrivals join behind it. Each train ahead has a finite sequence of actions before leaving. Only finitely many other approaches can occur before the head advances, so an infinite execution cannot keep postponing that advance. Together with deadlock freedom, this ensures eventual entry on every maximal execution, without assuming fairness between trains.

Availability (6–8)

From every reachable state, some continuation reaches `up` (6). This possibility does not mean that every execution opens the gate infinitely often: the stronger requirement (7) is false. Once the gate is closed, a train can repeatedly approach again immediately after leaving, before the controller chooses `raise`. The cycle

$$(\mathrm{approach}(t_1)\ .\ \mathrm{grant}(t_1)\ .\ \mathrm{enter}(t_1)\ .\ \mathrm{exit}(t_1)\ .\ \mathrm{leave}(t_1))^\omega$$

then keeps the gate closed forever. No train is denied entry in this execution, but road traffic is denied an opening. Guaranteeing recurring openings would require an additional controller policy or scheduling assumption; neither is part of the current model.

Non-vacuity (8) checks that, from every reachable state, each train can still reach `enter`, gate raising remains reachable, and some transition is enabled. These obligations apply throughout the system, rather than only in its initial state.

Broken Variant

The three faulty controllers are stored in `2/variants/` and checked with the same property files as the correct model. In the traces below, `lower` and `down` are shown for explanation; both are τ in the checked LTS.

Naive Raise

After a train leaves, the controller changes its recorded gate state to `opening` without checking the remaining queue or synchronising `raise` with the gate:

```
1  1      leave_controller(head(queue))                                  mcrl2
2     -    . CONTROLLER(tail(queue), free, gate)
   2  +    . CONTROLLER(tail(queue), free, opening)
```

The liveness property (5a) fails after the trace:

$$\text{approach}(t_1) \,.\, \text{lower} \,.\, \text{down} \,.\, \text{grant}(t_1) \,.\, \text{enter}(t_1) \,.\, \text{approach}(t_2) \,.\, \text{exit}(t_1) \,.\, \text{leave}(t_1)$$

The controller records `opening`, while the physical gate remains closed and waits for `raise`. The controller cannot offer `raise` in this state and cannot grant another train, since `grant` requires `closed`. Further trains may approach, but after they have all approached the system deadlocks. Properties 4, 5a, 6 and 8 therefore fail. Property 3 holds vacuously, because this faulty controller never performs `raise`.

Missed Increment

An approaching train is not added to the queue when another train has already been granted access. Approaches while the crossing is free are recorded normally:

```
1  1      approach_controller(t)                                         mcrl2
2     -    . CONTROLLER(queue <| t, crossing, gate)
   2  +    . CONTROLLER(if(crossing == busy, queue, queue <| t), crossing, gate)
```

The liveness property (5a) fails after:

$$\text{approach}(t_1) \,.\, \text{lower} \,.\, \text{down} \,.\, \text{grant}(t_1) \,.\, \text{enter}(t_1) \,.\, \text{approach}(t_2)$$

Train t_2 is now waiting for `grant`, but is absent from the queue and can never receive it. After t_1 exits and leaves, the controller can raise the gate despite t_2 still waiting, violating 3. Other trains can continue to cross repeatedly, violating 5b. Non-vacuity (8) also fails because entry for t_2 is no longer reachable.

This variant remains deadlock-free (4): a train that is already in the queue can finish crossing, and at least one train remains able to approach again when the crossing becomes free. Road availability (6) also holds, since the recorded queue can be emptied and the gate can still open. Thus a system can remain active while permanently excluding one train.

Off-by-One

When a train leaves, the controller removes it from the queue but incorrectly keeps the crossing marked `busy`:

```
1  1      leave_controller(head(queue))                                  mcrl2
2     -    . CONTROLLER(tail(queue), free, gate)
   2  +    . CONTROLLER(tail(queue), busy, gate)
```

The road availability property (6) fails after:

$$\text{approach}(t_1) \,.\, \text{lower} \,.\, \text{down} \,.\, \text{grant}(t_1) \,.\, \text{enter}(t_1) \,.\, \text{exit}(t_1) \,.\, \text{leave}(t_1)$$

Although the crossing is empty, both `grant` and `raise` are blocked by their requirement that it be `free`. A train approaching afterwards joins the queue but cannot be granted access. The controller can offer `leave` for the head, but that train is still waiting for `grant`, so the handshake cannot occur. Once every train has approached, the system deadlocks. Properties 4, 5a, 6 and 8 fail. Properties 1–3 and 5b hold; no further entry or opening can occur after the first departure.

The complete results are shown below, with T for `true` and F for `false`. Infinite opening (7) is false for the correct model as well as the faulty variants, so it does not distinguish these faults.

Model	1	2	3	4	5a	5b	6	7	8
Correct	T	T	T	T	T	T	T	F	T
Naive raise	T	T	T	F	F	T	F	F	F
Missed increment	T	T	F	T	F	F	T	F	F
Off-by-one	T	T	T	F	F	T	F	F	F

State Space and Abstraction

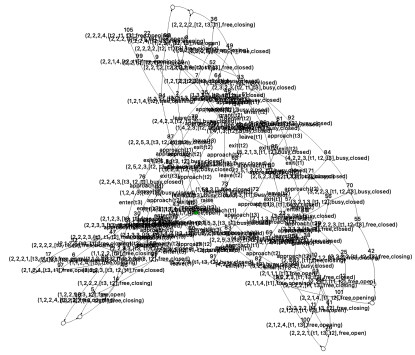


Figure 4: Full LTS

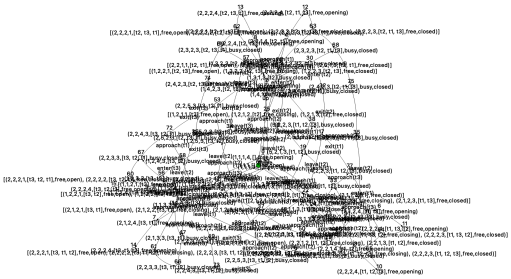


Figure 5: Reduced LTS

The figures show the three-train model. For $N = 1$ to 6 the full LTS (Figure 4) has 10, 31, 108, 451, 2278 and 13695 states, respectively. The reduced LTS (Figure 5) has 8, 23, 78, 323, 1628 and 9783. Each instance changes only the set of train IDs and the number of parallel train processes. Both LTSs grow quickly with N, because the trains interleave freely and the controller keeps a queue of waiting trains.

The Task 2 composition hides only the `lower` and `down` handshakes, leaving `grant`, `leave` and `raise` visible for verification. Branching bisimulation then merges states where these τ steps preserve the observable choices. For $N = 1$ to 6, the reductions are 20%, 25.81%, 27.78%, 28.38%, 28.53% and 28.57%, respectively. The proportion removed approaches 28.6% as N grows over these measurements.

All trains run the same process. Consistently renaming IDs in states and action labels preserves the transition structure and FIFO order. Reduction preserves this symmetry while keeping train identities distinct.

All results were recomputed with mCRL2 202607.0 using `python3 verify.py`, which also checks the witnesses, state counts and eventual entry on every maximal execution of the correct three-train model.

Conclusion

The FIFO controller is safe and live for trains. Faulty controllers can deadlock, exclude waiting trains or permit unsafe entry. Recurring gate openings for road traffic require an additional controller policy or scheduling assumption.